

Of Rehberi Web Uygulaması

PROJE RAPORU

Hazırlayan: Kubilay Kaan Durmuş

Öğrenci No: 25221601002

Danışman: Ahmet Gürhanlı

Ders: İleri Web Programlama (CME531)

Tarih: 16 Haziran 2026

Github: <https://github.com/kaan34-34/OfRehberiREACT>

1. Projenin Amacı

Of Rehberi projesi, yerel işletmelerin listelenebildiği, kullanıcıların firmaları inceleyip yorum yapabildiği, firmalar hakkında soru sorabildiği ve admin kullanıcısının işletme, yorum ve soru yönetimi yapabildiği tam yığın bir web uygulaması olarak geliştirilmiştir. Projenin temel amacı, modern web geliştirme yaklaşımını kullanarak React tabanlı bir ön yüz ile Spring Boot tabanlı katmanlı bir arka ucu gerçek API iletişimi üzerinden birleştirmektir.

Uygulama ilk halinde ön yüzde bulunan `mockData.js` dosyasındaki statik verilerle çalışırken, yapılan geliştirmeler sonucunda veri akışı tamamen backend REST API servislerine taşınmıştır. Bu sayede firmalar, kategoriler, yorumlar, kullanıcı işlemleri, soru-cevap kayıtları ve satın alma geçmişi veritabanı destekli hale getirilmiştir.

2. Projenin Hedefleri

Projenin hedefleri aşağıdaki şekilde belirlenmiştir:

- Katmanlı backend mimarisi kurmak: Controller, Service, Repository, Entity, DTO, Security ve Exception Handling yapıları ayrılmıştır.
- React + Material UI ile kullanıcı dostu bir arayüz geliştirmek.
- Firebase Authentication ile kullanıcı giriş/kayıt işlemlerini sağlamak.
- Firebase ID token bilgisini backend tarafında doğrulayarak Bearer token tabanlı yetkilendirme uygulamak.
- Kullanıcıların firmalara yıldızlı yorum eklemesini, kendi yorumlarını görüntülemesini ve silebilmesini sağlamak.
- Kullanıcıların firmalar hakkında soru sormasını, admin kullanıcıların bu sorulara cevap vermesini sağlamak.
- Admin panelinde firma ekleme, çoklu görsel yönetimi, firma düzenleme/silme, yorum moderasyonu ve kullanıcı mesajlarına cevap verme sekmelerini geliştirmek.
- Satın alma ve işlem geçmişi akışını backend servisleri üzerinden kayıt altına almak.
- Teslim klasör yapısını `SourceCode/frontend`, `SourceCode/backend` ve `SourceCode/database` şeklinde düzenlemek.

3. Kullanılan Teknolojiler

Ön yüzde React, JavaScript, Vite, Material UI, React Router, Axios ve Firebase Client SDK kullanılmıştır. React Router ile sayfa yönlendirmeleri yapılmış, Axios servis katmanı ile backend API çağrıları merkezi hale getirilmiştir. Firebase Client SDK, kullanıcı kayıt ve giriş işlemleri için kullanılmıştır.

Arka uçta Java 17, Spring Boot 3.3.7, Spring Web, Spring Data JPA, Spring Security, Bean Validation ve Firebase Admin SDK kullanılmıştır. Veritabanı bağlantısı geliştirme ortamında `localhost:3306`, `root/rootroot` bilgileriyle `ofrehberi` veritabanına yapılmaktadır. SQL şema ve başlangıç verileri `SourceCode/database` klasöründe tutulmuştur.

4. Sistem Mimarisi

Uygulama üç ana katmandan oluşur:

1. **Frontend:** Kullanıcının gördüğü React arayüzüdür. Sayfalar, bileşenler ve API servisleri bu bölümde bulunur.
2. **Backend:** REST API sunan Spring Boot uygulamasıdır. İş kuralları servis katmanında, veri erişimi repository katmanında, veri modelleri entity ve DTO sınıflarında tutulur.
3. **Database:** Firma, kategori, mahalle, yorum, kullanıcı, soru ve satın alma kayıtlarını içeren SQL tablolarından oluşur.

Frontend tarafındaki bütün veri istekleri `src/api` altındaki Axios servislerinden yapılmaktadır. `mockData.js` dosyası projeden kaldırılmış ve uygulamanın statik mock veriyle ilişkisi kesilmiştir. API çağrılarında Firebase oturumu varsa ID token otomatik olarak `Authorization: Bearer <firebase-id-token>` header bilgisiyle gönderilir.

Backend tarafında Controller sınıfları HTTP isteklerini karşılar, Service sınıfları iş kurallarını uygular, Repository sınıfları veritabanı işlemlerini yürütür. Hatalar `GlobalExceptionHandler` ile standart JSON cevaplarına dönüştürülür.

5. Backend Geliştirmeleri

Backend projesi `SourceCode/backend` klasörü altında Maven tabanlı Spring Boot projesi olarak oluşturulmuştur. Katmanlı mimari gereği aşağıdaki yapılar eklenmiştir:

- **Controller:** `BusinessController`, `AuthController`, `UserController`, `QuestionController`, `PurchaseController`, `AdminController`, `LookupController`
- **Service:** `BusinessService`, `AuthService`, `UserReviewService`, `QuestionService`, `PurchaseService`, `AdminReviewService`
- **Repository:** `BusinessRepository`, `ReviewRepository`, `AppUserRepository`, `QuestionRepository`, `PurchaseRepository`, kategori ve mahalle repository sınıfları
- **Entity:** `Business`, `Category`, `SubCategory`, `Neighborhood`, `Review`, `AppUser`, `Question`, `Purchase` ve ilgili detay/görsel entity sınıfları
- **DTO:** Firma özeti, firma detayı, yorum, kullanıcı, soru, satın alma ve lookup cevapları için record tabanlı DTO sınıfları
- **Security:** Firebase yapılandırması, CORS ayarları ve admin rol kontrolü
- **Exception Handling:** Kaynak bulunamadı, yetkisiz işlem ve validation hataları için merkezi hata yakalama

Backend API örnekleri:

- `GET /api/businesses`: Firma listesini döndürür.
- `GET /api/businesses/slug/{slug}`: Firma detayını, yorumları ve görselleri döndürür.
- `POST /api/auth/firebase-login`: Firebase ID token doğrular ve kullanıcıyı sisteme kaydeder/günceller.
- `GET /api/users/{userId}/reviews`: Kullanıcının geçmiş yorumlarını listeler.
- `POST /api/users/{userId}/businesses/{businessId}/reviews`: Kullanıcı adına yorum ekler.
- `DELETE /api/users/{userId}/reviews/{reviewId}`: Kullanıcının yorumunu siler.
- `POST /api/businesses/{businessId}/questions`: Firma için kullanıcı sorusu oluşturur.
- `PATCH /api/admin/questions/{questionId}/answer`: Admin cevabını kaydeder.
- `POST /api/admin/businesses`: Yeni firma ekler.
- `PUT /api/admin/businesses/{businessId}`: Firma bilgilerini ve çoklu görsel listesini günceller.
- `DELETE /api/admin/businesses/{businessId}`: Firmayı ve bağlı yönetim kayıtlarını siler.
- `GET /api/admin/reviews`: Tüm kullanıcı yorumlarını admin panelinde listeler.
- `DELETE /api/admin/reviews/{reviewId}`: Admin tarafından yorumu siler.

Firma puanı ve oy sayısı admin formundan alınmaz. `BusinessService` firma listesini veya detayını döndürürken ilgili `Review` kayıtlarını okuyarak ortalama yıldız puanını ve toplam yorum sayısını hesaplar. Böylece puan bilgisi manuel girişe değil, gerçek kullanıcı yorumlarına dayanır. Firma görselleri için ilk görsel `businesses.image_url` alanında ana görsel olarak tutulur, tüm görseller ise `business_gallery_images` tablosunda sıralı şekilde saklanır.

6. Güvenlik ve Yetkilendirme

Giriş ve kayıt işlemleri Firebase Authentication ile yapılmıştır. Kullanıcı e-posta ve şifre ile Firebase üzerinde doğrulanır. Başarılı girişten sonra frontend Firebase ID token üretir ve bu token backend'e gönderilir. Backend tarafında Firebase Admin SDK token bilgisini doğrular, kullanıcının Firebase UID ve e-posta bilgisini `app_users` tablosuna kaydeder.

Admin yetkisi iki seviyede kontrol edilmiştir. Birinci seviye frontend tarafındadır: kullanıcı `role=ADMIN` değilse admin paneli menüde görünmez ve `/admin` adresine doğrudan gidildiğinde kullanıcı giriş sayfasına yönlendirilir. İkinci seviye backend tarafındadır: admin endpointleri Bearer token doğrulaması yapar ve kullanıcının Firebase

admin claim bilgisi veya veritabanındaki **ADMIN** rolü yoksa işlem reddedilir. Böylece sadece arayüzde gizleme yapılmamış, gerçek işlem güvenliği backend üzerinde de uygulanmıştır.

7. Frontend Geliştirmeleri

React arayüzü **SourceCode/frontend** klasörü altında çalışmaktadır. Sayfa yapısı aşağıdaki temel bölümlerden oluşur:

- **HomePage:** Firma listesi, kategori/alt kategori filtreleri ve öne çıkan firmalar.
- **Top10Page:** Puan ve oy sayılarına göre öne çıkan firmalar.
- **BusinessDetailPage:** Firma bilgisi, yorumlar, yıldızlı yorum ekleme ve soru sorma alanı.
- **AuthPage:** Firebase e-posta/şifre ile giriş ve kayıt ekranı.
- **UserPanelPage:** Kullanıcının yorum ekleme, geçmiş yorumlarını görmesi ve silebilmesi.
- **AdminPage:** Firma ekleme/düzenleme, firma listeleme/silme, yorum moderasyonu ve kullanıcı sorularına cevap verme sekmeleri.

Veri erişimi için **src/api** klasörü eklenmiştir. **businessService**, **authService**, **userService**, **questionService** ve **adminService** dosyaları Axios üzerinden backend ile iletişim kurar. Axios interceptor, Firebase token bilgisini her isteğe otomatik ekler. Böylece bileşenler doğrudan mock veri okumak yerine API servislerini kullanır.

Admin panelinde **adminService** üzerinden firma oluşturma, firma güncelleme, firma silme, yorum listeleme, yorum silme, bekleyen soruları listeleme ve soruları cevaplama işlemleri yapılır. Firma formunda puan ve oy sayısı alanları kaldırılmıştır. Görsel alanı tek URL yerine dinamik liste olarak tasarlanmıştır; admin “Görsel Ekle” ile yeni URL alanı açabilir veya mevcut görsel URL satırını silebilir.

8. Kullanıcı İşlemleri

Kullanıcı sisteme ad soyad, e-posta ve şifre ile kayıt olabilir. Kayıt işlemi Firebase Authentication üzerinde oluşturulur; ardından backend bu kullanıcıyı kendi veritabanındaki **app_users** tablosuyla eşleştirir. Kullanıcı giriş yaptığında kullanıcı bilgisi local storage üzerinde tutulur ve arayüz bu bilgiye göre menüyü günceller.

Giriş yapan kullanıcı firma detay sayfasından yıldızlı yorum ekleyebilir. Kullanıcı paneli üzerinden geçmişte hangi firmalara yorum yaptığı görüntülenebilir ve kullanıcı kendi yorumlarını silebilir. Bu akış, kullanıcı deneyimini artırdığı gibi projenin kullanıcı bazlı veri yönetimi şartını da karşılar.

9. Soru-Cevap, Yorum Moderasyonu ve Admin Paneli

Kullanıcılar firma detay sayfasından firmalar hakkında soru sorabilir. Sorular backend’de **questions** tablosunda saklanır. Cevap bekleyen sorular admin panelindeki “Mesaj Cevapla” sekmesinde listelenir. Admin kullanıcı cevap yazdığında cevap veritabanına kaydedilir ve firma detay sayfasında kullanıcıya gösterilir.

Admin panelinde dört temel sekme yer alır:

- **Firma Ekle/Düzenle:** Yeni firma kaydı oluşturur veya seçili firmanın bilgilerini günceller. Firma ID, ad, slug, kategori, alt kategori, mahalle, iletişim, çalışma saati, özellikler ve çoklu görseller yönetilir. Firma puanı ve oy sayısı formdan girilmez; yorumlardan hesaplanır.
- **Firmalar:** Ekli firmaları listeler, düzenleme moduna alır veya siler. Düzenle seçildiğinde firma detayları ve galeri görselleri forma doldurulur.
- **Yorumlar:** Sitedeki tüm yorumları firma adı, kullanıcı adı, tarih ve yıldız bilgisiyle listeler. Admin uygunsuz yorumları silebilir.
- **Mesaj Cevapla:** Kullanıcı sorularına admin cevabı yazar.

Admin paneli role göre korunmuştur. Admin olmayan kullanıcı menüde bu sayfayı görmez ve backend üzerinde admin endpointlerini çalıştıramaz.

10. Satın Alma ve İşlem Geçmişi

Projede satın alma akışı backend servis katmanında oluşturulmuştur. `PurchaseService` ve `PurchaseController` üzerinden kullanıcıya bağlı satın alma kaydı üretilebilir ve kullanıcının geçmiş satın alma işlemleri tarih bilgisiyle listelenebilir. Mevcut geliştirme aşamasında gerçek Stripe PaymentIntent entegrasyonu yerine demo/mock checkout yaklaşımı kullanılmıştır. Bu yapı, ileride Stripe entegrasyonu eklendiğinde ödeme onayı alındıktan sonra aynı purchase tablosuna kayıt atılabilecek şekilde tasarlanmıştır.

11. Veritabanı Tasarımı

Veritabanı tabloları `SourceCode/database/schema.sql` içinde, başlangıç verileri `SourceCode/database/data.sql` içinde tutulur. Temel tablolar şunlardır:

- `categories`, `sub_categories`, `neighborhoods`
- `businesses`, `business_details`, `business_gallery_images`
- `reviews`
- `app_users`
- `questions`
- `purchases`

Başlangıçta frontend içinde bulunan veriler SQL seed verisine dönüştürülmüş, daha sonra frontend'in mock veri dosyası tamamen kaldırılmıştır. Uygulama artık verileri backend API üzerinden veritabanından almaktadır.

`businesses` tablosu firmanın temel bilgilerini, `business_details` detay bilgilerini, `business_gallery_images` ise çoklu görsel URL listesini tutar. `reviews` tablosu yıldızlı yorumları içerir; firma puanı ve oy sayısı bu tablodaki kayıtların ortalaması ve toplam sayısı üzerinden hesaplanır. Admin yorum sildiğinde puan hesaplaması da otomatik olarak güncel yorum setine göre değişir.

12. Projeyi Çalıştırma Adımları

4. Proje ana klasöründe gerekli frontend paketleri kurulur:

```
npm install
```

5. `SourceCode/frontend/.env` dosyasına Firebase web app bilgileri yazılır. Örnek dosya `SourceCode/frontend/.env.example` içinde bulunur.
6. Backend Firebase Admin servis hesabı şu klasöre eklenir:

```
SourceCode/backend/firebase-admin/
```

7. MySQL veritabanı bilgileri `SourceCode/backend/src/main/resources/application.properties` içinde yer alır:

```
spring.datasource.url=jdbc:mysql://localhost:3306/ofrehberi
spring.datasource.username=root
spring.datasource.password=rootroot
```

8. Proje tek komutla çalıştırılır:

```
npm start
```

Bu komut önce backend Maven build işlemini yapar, Spring Boot uygulamasını <http://localhost:8080> üzerinde başlatır, ardından Vite frontend uygulamasını <http://localhost:5173> üzerinde açar.

9. Admin kullanıcı oluşturmak için:

```
cd SourceCode/backend/firebase-admin
node make-admin.js kullanıcı@mail.com
```

Admin claim verildikten sonra kullanıcının çıkış yapıp tekrar giriş yapması gerekir.

13. Test ve Doğrulama Sonuçları

Geliştirme sürecinde aşağıdaki kontroller yapılmıştır:

- Backend Maven build komutu başarıyla çalıştırılmıştır.
- Frontend Vite build komutu başarıyla çalıştırılmıştır.
- Backend CORS ayarları `localhost:5173` frontend adresi için doğrulanmıştır.
- `Authorization` header bilgisinin preflight isteğinde izinli olduğu kontrol edilmiştir.
- `mockData.js` dosyasına ait import ve referansların tamamen kaldırıldığı `rg` aramasıyla doğrulanmıştır.
- Kullanıcı girişinde yanlış şifre durumunda Firebase hatası, doğru girişte backend kullanıcı eşleştirmesi ayrıştırılmıştır.
- Canlı veritabanında `app_users.id` alanı `AUTO_INCREMENT` yapılarak Firebase kullanıcılarının backend'e doğru kaydedilmesi sağlanmıştır.
- Admin panelinin sadece `ADMIN` rolündeki kullanıcılar tarafından görülebilmesi ve kullanılabilmesi sağlanmıştır.
- Admin tarafında firmaların listelenmesi, düzenlenmesi, silinmesi ve tüm yorumların görüntülenip silinmesi doğrulanmıştır.
- Firma ekleme/düzenleme formunda puan ve oy sayısı girişlerinin kaldırıldığı, çoklu görsel URL ekleme/silme akışının çalıştığı kontrol edilmiştir.
- `GET /api/admin/reviews` endpointinin token olmadan `403 Forbidden` döndürdüğü ve public firma API'sinin `200 OK` döndüğü doğrulanmıştır.

14. Sonuç

Of Rehberi projesi, statik veriyle çalışan bir React uygulamasından katmanlı backend mimarisine sahip, Firebase Authentication kullanan, API tabanlı ve rol kontrollü bir web uygulamasına dönüştürülmüştür. Kullanıcı kayıt/giriş, yıldızlı yorum, kullanıcı paneli, soru-cevap, firma CRUD yönetimi, yorum moderasyonu, çoklu görsel yönetimi, admin paneli ve işlem geçmişi gibi özellikler projeye eklenmiştir.

Bu çalışma sonucunda frontend ve backend arasında gerçek REST API iletişimi kurulmuş, güvenlik Bearer token doğrulamasıyla güçlendirilmiş ve proje final teslim yönergesindeki `SourceCode/frontend`, `SourceCode/backend`, `SourceCode/database` klasör yapısına uygun hale getirilmiştir. Proje, ileri web programlama kapsamında modern web uygulamalarında beklenen katmanlı mimari, kimlik doğrulama, yetkilendirme, veri yönetimi ve kullanıcı/admin ayrımı konularını uygulamalı olarak göstermektedir.